

GITHUB TEAM WORK GUIDE

GITHUB TEAM WORK GUIDE is a guide repo. This README file provides guidelines on setting up the project, branching strategy, commit message conventions, and contribution rules.

Getting Started

1 Clone the Repository

```
git clone <https://github.com/your-repo/workshop-2.git>
cd workshop-git
```

2 Install Dependencies

```
npm install
```

3 Start the Development Server

```
npm run dev
```

The application should now be running at <http://localhost:5173>.

Branching Strategy

We follow the **Git Flow** branching model:

- `main` → **Production** (Only stable and tested code)
- `develop` → **Development Branch** (Latest features being worked on)
- `username/pagename` → **UI pages** (If you working on a page)
- `feature/{feature-name}` → **Feature Branches** (For new features)
- `fix/{bug-name}` → **Bug Fixes**
- `docs/{documentation-portion}` → **Documentation Updates**

Example:

```
masud-parvez/homepage  
feature/user-authentication  
fix/navbar-bug  
docs/readme-update
```

Commit Message Convention

We follow the **Conventional Commits** format:

Structure:

```
<type>(scope): <subject>
```

Example:

```
design(homePage) : complete Home Page Design  
feat(auth): add JWT authentication system  
fix(ui): resolve navbar overlap issue on mobile  
docs(readme): update installation steps
```

Allowed Commit Types:

- **feat** → For new features
- **fix** → For bug fixes
- **docs** → For documentation updates
- **style** → Code formatting (no logic changes)
- **refactor** → Code restructuring (no functionality changes)

Contribution Rules

Before pushing any changes, follow these steps:

1 Pull the latest changes before starting work:

```
git pull origin develop
```

2 Create a new branch based on the pages or features or fix:

```
git checkout -b md-mehedi-hasan-talha/homePage
```

3 Follow coding standards (Prettier & ESLint):

```
npm run lint
```

4 Write meaningful commit messages using the defined convention.

5 Create a Pull Request (PR) for merging into `develop`.

6 Wait for at least one approval before merging.

! Important Rules

✓ Always write **clean and modular** code.

✓ Use

meaningful variable names.

✓ Never push directly to

`main` or `develop`.

✓ Always get code reviewed before merging.

✓ Use

environment variables for sensitive information.

Contact & Support

For any issues, reach out to the project maintainers. `Masud` and `Talha`

Happy Coding! 🚀🎉